

## Table of Contents

|                                                         |   |
|---------------------------------------------------------|---|
| 1. Asynchronous Database Access with MongoDB            | 1 |
| 2. Encapsulation and Object-Oriented Structure          | 2 |
| 3. Dynamic UI Updates with TextMeshPro(TMP)             | 2 |
| 4. Persistent Storage with PlayerPrefs                  | 3 |
| 5. Data Validation and Preemptive Error Handling        | 3 |
| 6. Drift Simulation Using Physics and Slip Detection    | 4 |
| 7. Loops and Collections for Scalable Component Control | 4 |
| 8. Customizable Key Input for Features                  | 5 |
| 9. Speedometer and Gear Visual Feedback                 | 5 |
| 10. Wheel Rotation Sync Between Visuals and Physics     | 6 |

## 1. Asynchronous Database Access with MongoDB

**Technique:** Advanced Database

**Script:** *LoginRegisterManager.cs* / Lines 20 -108

```
void Start()
{
    // connect to mongodb (CHANGE PASSWORD)
    string connectionString =
"mongodb+srv://Learn2Drive:Learn2Drive@cluster0.z6neyyd.mongodb.net/?re
tryWrites=true&w=majority";
    client = new MongoClient(connectionString);

public async void OnRegisterClicked()
{
    // validation system response via debug log or console
    string username = usernameInput.text.Trim();
```

```

        string password = passwordInput.text;

        if (string.IsNullOrEmpty(username) ||
string.IsNullOrEmpty(password))
        {
            Debug.LogWarning("username or password cannot be empty");
            return;
        }

        if (!IsPasswordStrong(password))
        {
            Debug.LogWarning("password must be at least 8 characters
with uppercase, lowercase, number, and special symbol");
            return;
        }

        var existingUser = await
userCollection.Find(Builders<BsonDocument>.Filter.Eq("username",
username)).FirstOrDefaultAsync();

        if (existingUser != null)
        {
            Debug.LogWarning("user already exists");
            return;
        }

        var newUser = new BsonDocument
        {
            { "username", username },
            { "password", password }
        };

        await userCollection.InsertOneAsync(newUser);
        Debug.Log("user registered successfully");

        SceneManager.LoadScene("Main Menu");
    }

    public async void OnLoginClicked()
    {
        string username = usernameInput.text.Trim();
        string password = passwordInput.text;
    }

```

```

        var filter = Builders<BsonDocument>.Filter.And(
            Builders<BsonDocument>.Filter.Eq("username", username),
            Builders<BsonDocument>.Filter.Eq("password", password)
        );

        var user = await
userCollection.Find(filter).FirstOrDefaultAsync();

        if (user != null)
        {
            Debug.Log("login successful");
            SceneManager.LoadScene("Main Menu");
        }
        else
        {
            Debug.LogWarning("invalid username or password");
        }
    }
}

```

## Appropriateness

Using asynchronous database calls (**async** / **await**) for registration and login prevents the Unity main thread from freezing while network requests complete. For a learning app that must remain responsive during short study sessions non-blocking I/O ensures UI elements such as menus and feedback messages to continue to work while data is read or written. In line with the success criteria requiring a “working login/signup system,” this approach avoids interruptions and aligns with the client’s desire for a platform That feels professional and polished just like the online tools she’s used to.

## Limitations

The current implementation exposes two critical security concerns, passwords are saved in plaintext in the database and the database connection string (including credentials) is embedded client-side. Client-side storage of credentials makes them accessible to anyone who inspects the build and plaintext password storage risks user privacy and account takeover. Additionally **async void** methods provide poor error propagation and are difficult to test and/or cancel.

## Improvements:

- More authentication behind a secure server-based API. The unity client should send login requests to an HTTPS endpoint where the server itself handles DB access
- Hash and salt passwords on the server (use bcrypt) and store only hashes.

- Never embed DB credentials in the client rather use environment variables or cloud-managed secrets on the server
- 

## 2. Encapsulation and Object-Oriented Structure

**Technique:** Java Coding

**Scripts:** All scripts (e.g., *Car\_Controller.cs*, *Radio.cs*)

```
public class Car_Controller : MonoBehaviour
{
    //Public Variables
    [Header("Wheel Colliders")]
    public List<WheelCollider> Front_Wheels; //The front wheels
    public List<WheelCollider> Back_Wheels; //The rear wheels

    [Header("Car Settings")]
    public float Motor_Torque = 400; //Motor torque for the car
    public float Max_Steer_Angle = 25f; //The Maximum Steer Angle for
the front wheels
    public float BrakeForce = 150f; //The brake force of the wheels
    public float Maximum_Speed; //The top speed of the car

    void Start() {
        //To Prevent The Car From Toppling When Turning Too Much
        rb = GetComponent<Rigidbody>(); //get rigidbody
        rb.centerOfMass = Center_of_Mass.localPosition; //Set the
centre of mass of the rigid body to the centre of mass transform
        public void FixedUpdate(){
            foreach(WheelCollider Wheel in Front_Wheels){
                Wheel.steerAngle = Input.GetAxis("Horizontal") *
Max_Steer_Angle; //Turn the wheels
            }

            public void Boost_Function(){
                rb.AddForce(Boost_Amount * transform.forward,
ForceMode.VelocityChange);
```

The program is structured using OOP principles where each major feature of the simulation is enveloped within its own class. For example in this code audio systems and vehicle behavior are handled by separate scripts rather than being combined into a single controller. This approach is appropriate for improving maintainability and scalability allowing features to

be modified or extended independently without affecting unrelated components. However it can be ineffective if classes are poorly scoped or excessively interconnecting through the overuse of public variables which risks exposing internal state and increasing coupling between components which can make debugging more difficult. To improve this technique clearer separation of responsibilities should be enforced by reducing unnecessary public fields and using controlled access methods where appropriate.

---

### 3. Dynamic UI Updates with TextMeshPro(TMP)

**Technique:** GUI (Java Swing Equivalent)

**Script:** *Radio.cs* / Lines 57 -81

*//Setting the UI components*

```
        if(Use_UI){
            //Set the Song name to the text object
            Song_Name_Text.SetText(Tracks[Current_Track].ToString());

            //Set the song number (from the list) to the text object
            Song_Number_Text.SetText((Current_Track + 1).ToString());
        }
    }

    public void Next_Song(){
        //If the current track is the last one go back to the 1track
        if(Tracks.Last() == Tracks[Current_Track]){
            Current_Track = 0;
            Audio_Source.enabled = false;
            Audio_Source.clip = Tracks[Current_Track];
            Audio_Source.enabled = true;
        }

        //If the current track is not the last one go to the next one
        else{
            Current_Track++;
            Audio_Source.enabled = false;
            Audio_Source.clip = Tracks[Current_Track];
            Audio_Source.enabled = true;
        }
    }
}
```

TMP is used to update on-screen text dynamically such as displaying speed or system feedback. This technique is appropriate because it enables clear, real-time communication with the user which is essential in a learning focused application where feedback must be readable, immediate and relevant. A limitation of this approach is that updating UI elements too frequently such as every frame can introduce unnecessary performance overhead where if values are repeatedly written even when unchanged may lead to inefficient memory usage especially on lower-end devices. This problem could be improved by implementing conditional updates so that the text is only refreshed when the underlying values change with reduces redundant processing whilst maintaining responsiveness.

---

## 4. Persistent Storage with PlayerPrefs

**Technique:** Database / File Handling

**Script:** *CarSelector.cs* / Lines 10, 38 -43

**Why it was used:** To store and recall user selections like car types between sessions.

**Explanation:**

PlayerPrefs allow Unity developers to store key-value data:

```
private void Awake()
{
    int savedCar = PlayerPrefs.GetInt("SelectedCar", 0); // Load
previous selection
    SelectCar(savedCar);
}
public void ConfirmCarSelection()
{
    PlayerPrefs.SetInt("SelectedCar", currentCar); // Save selected
car
    PlayerPrefs.Save();
    SceneManager.LoadScene("Map Selection"); // Load the Map
Selection Scene
}
```

PlayerPrefs is used to store simple user preferences such as selected vehicles between sessions. This technique is appropriate because it offers a lightweight and easily accessible way to persist non-sensitive data without requiring external databases to ensure user continuity allowing them to resume the application without the reconfiguration process

restarting. However Playerprefs is not secure and stores data locally in a format that can be modified by the user which makes it unsuitable for sensitive or critical data such as authentication details or progression records. To improve this Playerprefs could remain limited to strictly non-sensitive data configuration where a secure server-side storage would be implemented for the sensitive data.

---

## 5. Data Validation and Preemptive Error Handling

**Technique:** Error Trapping / Algorithmic Logic

**Script:** *LoginRegisterManager.cs* / Lines 35 -39

**Why it was used:** To prevent empty or invalid input from being processed.

**Explanation:**

Before submitting login or signup requests, the app checks:

```
public async void OnRegisterClicked()
{
    // validation system response via debug log or console
    string username = usernameInput.text.Trim();

    string password = passwordInput.text;

    if (string.IsNullOrEmpty(username) ||
string.IsNullOrEmpty(password))
    {
        Debug.LogWarning("username or password cannot be empty");
        return;
    }

    if (!IsPasswordStrong(password))
    {
        Debug.LogWarning("password must be at least 8 characters
with uppercase, lowercase, number, and special symbol");
        return;
    }
}
```

The program validates user input by checking for empty fields and enforcing password complexity requirements allowing database interactions which improves usability through

immediate feedback and reduces the likelihood of runtime errors caused by malformed input. Despite its usefulness it doesn't provide security as users can bypass these checks by modifying the client in addition to a lack of encryption and security processes. To improve this technique, validation should be mirrored server-side to ensure data integrity regardless of client behavior.

---

## 6. Drift Simulation Using Physics and Slip Detection

**Technique:** Algorithmic Logic / Physics

**Script:** *Car\_Controller.cs* / Lines 304 -370

**Why it was used:** To simulate realistic tire grip under different driving conditions.

**Explanation:**

```
//Make Car Drift
WheelHit wheelHit;

foreach(WheelCollider Wheel in Back_Wheels){
    Wheel.GetGroundHit(out wheelHit);

    if(wheelHit.sidewaysSlip < 0 )

tempo = (1 + -Input.GetAxis("Horizontal")) *
Mathf.Abs(wheelHit.sidewaysSlip *handBrakeFrictionMultiplier);

    if(tempo < 0.5) tempo = 0.5f;

    if(wheelHit.sidewaysSlip > 0 )
        tempo = (1 + Input.GetAxis("Horizontal")) *
Mathf.Abs(wheelHit.sidewaysSlip *handBrakeFrictionMultiplier);

    if(tempo < 0.5) tempo = 0.5f;

    if(wheelHit.sidewaysSlip > .99f || wheelHit.sidewaysSlip < -.99f){
        //handBrakeFriction = tempo * 3;
        float velocity = 0;
        handBrakeFriction =
Mathf.SmoothDamp(handBrakeFriction,tempo* 3,ref velocity ,0.1f *
Time.deltaTime);
    }
}
```

```

        else{
            handBrakeFriction = tempo;
        }
    }
}

```

The simulations models drifting behaviour as a indicator of a loss of traction by analysing wheel slip values from Unity's physics system and applying funtions to adjust the traction. The main limitation is that it simplifies real tyre dynamics as slip values offer only an approximation and do not fully account for factors such as weight, load sensitivity or tempurature.

---

## 7. Loops and Collections for Scalable Component Control

**Technique:** Java Coding - Loops

**Script:** *Car\_Controller.cs* / Lines 91 -98

*//Here we turn the reverse light(s) off*

```

if(Enable_Reverselights_Lights){
    foreach(Light R in ReverseLights){
        R.enabled = false;
    }
}

```

The program uses collections and loops to manage groups of similar components such as lights or wheels where it allows the progrma to scale easily through adding or removing components dthat do not require rewriting logic. However iterating through collections frequently especially frame-by-frame may reduce performance if the number of elements increases significcantly where in addition a misuse of looping constructs in performance-criticla contexts may cause unnecessary memory allocations. To improve this updates should be limited to when the state itself changes and for larger collections groupign elements under parent objects or using pooling techniques may be more efficient.

---

## 8. Customizable Key Input for Features

**Technique:** GUI / Interaction

**Scripts:** *Car\_Controller.cs*, *Radio.cs*

**Why it was used:** To allow users to interact with the simulation using keys of their

choosing.

**Explanation:**

```
//Make Car Boost
    if(Input.GetKeyDown(Boost_KeyCode) && Car_Started &&
Next_Boost_Time < Time.time){
        //BOOST CAR
        Boost_Function();
        Next_Boost_Time = Time.time + Boost_Cooldown; //The
cooldown for the car
    }
```

User input is handled using configurable key bindings allowing actions such as boosting to be triggered by assigned keys. A limitation is that key bindings are currently defined in code and not adjustable through user interface which reduces accessibility for non-technical users. Improvements include the implementation of an in-game settings menu that allows users to remap controls dynamically and adopting a more advanced input system would also support multiple devices and improve extensibility.

---

## 9. Speedometer and Gear Visual Feedback

**Technique:** GUI

**Script:** *Speedometer\_Controller.cs* / Lines 33 -71

**Why it was used:** To show driving data (speed and gear) in a car-like interface.

**Explanation:**

```
//If speed_text var is assigned
    if (Speed_Text != null){
        //Change text of the Text var to the speed
        Speed_To_Display = (int) speed;
        Speed_Text.SetText(Speed_To_Display.ToString());
    }

    //if arrow rect var is assigned
    if (Arrow != null){
        //Rotate the arrow
```

```

        Arrow.localEulerAngles = new Vector3(0, 0,
Mathf.Lerp(Minimum_Speed_Arrow_Angle, Maximum_Speed_Arrow_Angle, speed
/ Maximum_Speed));
    }

    //do this if the speed is NOT equal to 0
    if(!speed_is_zero){
        //Setting the gear text to the current gear
        Gear_Text.SetText(Current_Gear);
    }
}

```

The speedometer system visually represents vehicle speed and gear using animated UI elements to mirror real-world vehicle dashboards reinforcing learning through visual cues. One limitation is that without clear documentation and structure the relationship between engine speed and vehicle speed is not visually defined or coded included in the improvement would be speed mapping and the addition for more familiar guages.

---

## 10. Wheel Rotation Sync Between Visuals and Physics

**Technique:** Algorithmic Thinking

**Script:** *Car\_Controller.cs* / Lines 590-607

**Why it was used:** To visually align wheels with physics-based wheel colliders.

**Explanation:**

```

//Rotating The Wheels Meshes so they have the same position and
rotation as the wheel colliders
var pos = Vector3.zero; //position value (temporary)
var rot = Quaternion.identity; //rotation value (temporary)

for (int i = 0; i < (Back_Wheels.Count); i++)
{
    Back_Wheels[i].GetWorldPose(out pos, out rot); //get the
world rotation & position of the wheel colliders
    Back_Wheel_Transforms[i].position = pos; //Set the wheel
transform positions to the wheel collider positions

```

```
        Back_Wheel_Transforms[i].rotation = rot *  
Quaternion.Euler(Back_Wheel_Rotation[i]); //Rotate the wheel transforms  
to the rotation of the wheel collider(s) and the rotation offset  
    }
```

The program synchronizes visual wheel models with physics colliders to ensure consistency between simulation behavior and on screen representation as mismatches between physics and visuals can confuse users. Limitations arise if model orientations or collider are disabled which may cause misalignment. To improve this clear documentation of any offsets and handling edge cases explicitly in addition to verification procedures comparing expected and actual positions.

Word Count: 978